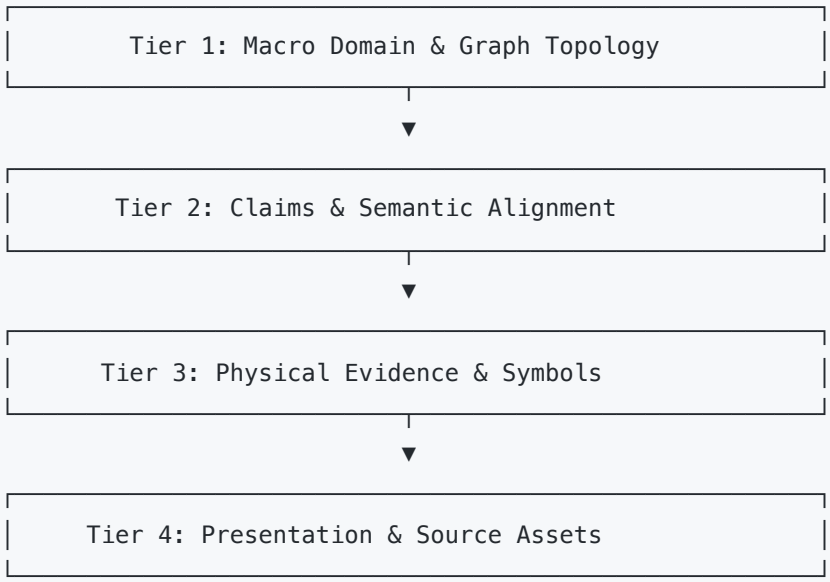
OntoForge: Systems Architecture & Engineering Design Specification (v1.0)

This specification defines the core internals, protocols, and data schemas for **OntoForge**—a local-first, extensible, schema-driven Integrated Research Environment (IRE) designed to treat *Papers as Code*.

---

1. Multi-Tier Architecture & Cognitive Decoupling

OntoForge isolates uncertainty by enforcing **Layered Assumptions**. Each layer assumes the layer beneath it is 100% deterministic, while exposing specialized Human-in-the-Loop (HITL) error-correction boundaries.



1.1 Tier 1: Macro Domain & Graph Topology (Strategic Intent)

- **Responsibility:** Manages taxonomy trees (DAGs), technical evolution milestones, and cross-paper structural relationships.
- **Pluggability Rule:** Tier 1 does not hardcode ontology trees or metric lists in source code. It loads them from an external, versioned Domain Pack config file at bootstrap.
- **Downwards Assumption:** Assumes all scientific claims and assertions passed from Tier 2 are mathematically clean and fully semantic-aligned.
- **HITL Boundary:** Triggered when structural changes create macro-logical impossibilities (e.g., historical temporal inversion or baseline regression). Handled via a **Git-like Merge Conflict**

Interface.

- **Intent Framework Integration:** All Tier 1 HITL actions captured as **Priority** or **Constraint** intents (Section 2.6). Conflict resolutions build learned preference patterns for evidence ranking. Assertion locks create absolute constraints that shape future AI behavior.

1.2 Tier 2: Claims & Semantic Alignment (The Matrix View)

- **Responsibility:** Normalizes heterogeneous scientific statements into strictly typed assertions bound by explicit performance metrics and environmental constraints.
- **Downwards Assumption:** Assumes that raw text parsing, variable declarations, and tabular matrices from Tier 3 are completely free of OCR/transcription errors.
- **HITL Boundary:** Triggered when the AI cannot resolve if two terms (e.g., “Latency” vs. “Propagation Delay”) represent the same coordinate. Handled via the **Global Alignment Matrix Editor**.
- **Intent Framework Integration:** Unit selections and metric alignments captured as **Preference** intents with **Pattern** scope (Section 2.6). System learns context-specific preferences (e.g., “use [ms] for reconfiguration_delay in optical networking domain”) and proactively suggests units in future extractions.

1.3 Tier 3: Physical Evidence & Symbols (The Ground Truth)

- **Responsibility:** Parses arbitrary two-dimensional layouts (PDFs), normalizes math equations into LaTeX, maps out multi-modal structures (figures/tables), and links every token to absolute geometric pixel coordinates (`bounding_boxes`).
- **Downwards Assumption:** Assumes Tier 4 provides stable source artifacts and that the underlying VLM/Layout parser can extract structural tokens perfectly from that presentation layer.
- **HITL Boundary:** Triggered when layout parsing or LaTeX syntax validation falls below safety thresholds. Handled via **In-Line Visual Squiggles** and the **Digital Microscope view**.
- **Intent Framework Integration:** OCR/LaTeX corrections captured as **Correction** intents with **Local** scope (Section 2.6). Limited generalization potential, but the system can learn layout patterns (e.g., “two-column IEEE format has known parsing quirks”) to improve Visual Fallback Alert accuracy.

1.4 Tier 4: Presentation & Source Assets (PDF / Visual Surface)

- **Responsibility:** Owns the human-visible source artifacts exactly as imported, including PDFs, page images, embedded figures, and view-layer document metadata.
- **Pluggability Rule:** Tier 4 must preserve original source formats without forcing early semantic conversion, so future importers can support other presentation containers beyond PDF.
- **Downwards Assumption:** Assumes only that the source asset is a faithful presentation surface worth preserving, not that its internal structure has yet been parsed correctly.
- **HITL Boundary:** Triggered when imported presentation artifacts are incomplete, visually corrupted, or mismatched against expected provenance. Handled via source reattachment, page-level replacement, and visual inspection workflows.

2. Scientific Service Protocol (SSP v1.0) Specification

The SSP protocol decouples the OntoForge Client IDE from language-specific or domain-specific backend parsing engines. Communication is driven over standard JSON-RPC 2.0.

2.0 Workspace & Multi-Domain-Pack Architecture

Workspace is OntoForge's top-level organizational unit representing a bounded research project. Each workspace:

- **Contains multiple domain packs** - Supporting interdisciplinary research that spans domains (e.g., comparing optical vs wireless technologies)
- **Has a unique identity** - Workspace name, creation timestamp, and metadata
- **Isolates research context** - Papers, assertions, conflicts, and intent profiles are scoped to the workspace
- **Maintains namespace separation** - Metrics from different domain packs use namespace prefixes to avoid collisions

Architecture Benefits:

```
Workspace: "5G Backhaul Comparison Study"
├── 📦 Domain Pack: Optical Access Networks (namespace: optical)
│   ├── 📄 Papers (12)
│   ├── 🏷️ Subdomains (WDM, FSO, OPS)
│   └── 📊 Metrics (reconfiguration_delay, cross_talk, insertion_loss, power_consumption)
│
└── 📦 Domain Pack: Wireless Communications (namespace: wireless)
    ├── 📄 Papers (8)
    ├── 🏷️ Subdomains (mmWave, Sub-6GHz)
    └── 📊 Metrics (throughput, latency, coverage_radius, energy_efficiency)
```

Namespace Isolation:

Metrics are namespaced to prevent collisions:

- `optical:reconfiguration_delay` vs `wireless:latency`
- Assertions reference fully-qualified metric names
- Matrix View can filter by active domain pack(s)
- Conflicts only detected within same domain pack

Multi-Workspace Support:

Users can maintain multiple workspaces for different research projects:

```
~/research/
├── 5g_backhaul_study/        (2 domain packs: optical + wireless)
├── quantum_networking/        (1 domain pack: quantum)
└── ml_for_networks/           (2 domain packs: networking + ml)
```

2.1 `textDocument/sspInitialize`

- **Direction:** Client $\rightarrow$ Server
- **Intent:** Initialize or load a workspace with one or more domain packs. The workspace serves as the container for all papers, assertions, and research context.
- **Request Payload:**


```
{
  "jsonrpc": "2.0",
  "method": "textDocument/sspInitialize",
  "params": {
    "workspace": {
      "id": "workspace_5g_backhaul_2026",
      "name": "5G Backhaul Comparison Study",
      "description": "Comparing optical and wireless backhaul technologies for 5G networks",
      "root": "file:///Users/scholar/research/5g_backhaul_study"
    },
    "domain_packs": [
      {
        "namespace": "optical",
        "config_ref": {
          "path": "./configs/optical-access-networks.yaml",
          "format": "yaml",
          "watch": true
        },
        "enabled": true
      },
      {
        "namespace": "wireless",
        "config_ref": {
          "path": "./configs/wireless-communications.yaml",
          "format": "yaml",
          "watch": true
        },
        "enabled": true
      }
    ],
    "bootstrap_mode": "workspace-first"
  },
  "id": 1
```

```
}
```

Response Payload:

```
{
  "jsonrpc": "2.0",
  "result": {
    "workspace_id": "workspace_5g_backhaul_2026",
    "status": "initialized",
    "loaded_domain_packs": [
      {
        "namespace": "optical",
        "domain_id": "domain.networking.optical",
        "name": "Optical Access Networks",
        "version": "1.0",
        "metrics_count": 4,
        "subdomains_count": 3
      },
      {
        "namespace": "wireless",
        "domain_id": "domain.networking.wireless",
        "name": "Wireless Communications",
        "version": "1.0",
        "metrics_count": 5,
        "subdomains_count": 2
      }
    ],
    "total_papers": 20,
    "ready": true
  },
  "id": 1
}
```

2.1.1 External Domain Pack Config (`domain-pack.yaml`)

OntoForge treats domain ontology and core metrics as **pluggable runtime inputs**, not compile-time constants. Each domain pack is loaded into a workspace with a unique namespace to prevent metric collisions.

- **Bootstrap Source of Truth:** `config_ref.path` in workspace initialization
- **Namespace Isolation:** All metrics prefixed with namespace (e.g., `optical:reconfiguration_delay`)
- **Hot Reload:** If `watch=true` , config changes trigger revalidation and update in-memory registries
- **Compatibility Gate:** Config must include `schema_version` ; loader rejects unknown major versions

Minimal YAML Shape:

```yaml
schema_version: 1.0
domain:
  id: domain.networking.optical
  name: Optical Access Networks
  parent_domain_id: null  # Top-level domain
  taxonomy_tree:
    - name: topology
      values: [mesh, ring, star, free_space]
      subdomain_map:
        mesh: domain.networking.optical.access
        free_space: domain.networking.optical.fso
      description: Network topology architectures
    - name: switching_mechanism
      values: [wdm, optical_packet, burst_switching]
      subdomain_map:
        wdm: domain.networking.optical.wdm
        optical_packet: domain.networking.optical.ops
        burst_switching: domain.networking.optical.obs
      description: Switching technologies and methods

subdomains:
  - id: domain.networking.optical.wdm
    name: Wavelength Division Multiplexing
    parent_domain_id: domain.networking.optical
    description: WDM-based optical switching systems
    classification_hints:
      keywords: [wdm, wavelength, division, multiplexing, dwdm, cwdm]
  - id: domain.networking.optical.fso
    name: Free-Space Optical
    parent_domain_id: domain.networking.optical
    description: Free-space optical communication systems
    classification_hints:
      keywords: [free-space, fso, atmospheric, satellite, optical wireless]

core_metrics:
  - name: reconfiguration_delay
    expected_unit: ms
    aliases: [switching_delay, matrix_latency]
    value_type: float
  - name: cross_talk
    expected_unit: dB
    aliases: [crosstalk]
    value_type: float

validation:
```

```
  required_context_keys: [network_load, topology]
  unit_normalization:
    ms: [ms, millisecond, milliseconds]
    dB: [dB, db]
```

Loader Behavior:

1. Parse YAML/JSON and validate against a Pydantic Domain Pack schema.
2. Build immutable in-memory registries: `metric_registry`, `alias_registry`, `unit_registry`, `subdomain_registry`.
3. Extract `TaxonomyDimension` objects with subdomain mappings from `taxonomy_tree`.
4. Persist a `config_hash` and `domain_pack_id` in each ingest session for reproducibility.
5. Reject extraction outputs whose `subject` is not in `metric_registry` unless manually whitelisted by HITL.
6. Validate subdomain IDs in `subdomain_map` against declared subdomains section.

2.2 `textDocument/sspIngest`

- **Direction:** Client $\rightarrow$ Server (Asynchronous/Streamable)
- **Intent:** Strip narrative filler from a newly imported PDF file, extract atomic knowledge tensors, classify into subdomains using taxonomy-based AI classification, and return deterministic spatial anchors. Papers are ingested into a specific workspace and can reference metrics from any loaded domain pack.
- **Request Payload:**

```
{
  "jsonrpc": "2.0",
  "method": "textDocument/sspIngest",
  "params": {
    "workspace_id": "workspace_5g_backhaul_2026",
    "target_domain_pack": "optical",
    "paper_uri": "file:///Users/scholar/papers/satellite_switching_2026.pdf",
    "paper_id": "paper_satellite_switching_2026",
    "parse_options": {
      "extract_figures": true,
      "extract_tables": true,
      "confidence_threshold": 0.7,
      "auto_classify": true
    }
  },
  "id": 2
}
```

- **Response Payload:**


```
{
  "jsonrpc": "2.0",
  "result": {
    "paper_id": "paper_satellite_switching_2026",
    "metadata": {
      "title": "On-Board Optical Packet Switching for Mega-Constellations",
      "authors": ["Zhang, X.", "Li, Y."],
      "year": 2026,
      "venue": "OFC 2026"
    },
    "extracted_chunks": [
      {
        "chunk_id": "chunk_p4_c2",
        "raw_text": "The experimental setup achieves a total matrix reconfiguration latency o
        "bounding_box": {"page": 4, "coords": [100, 200, 450, 280]}
      }
    ],
    "extracted_formulas": [
      {
        "formula_id": "form_eq_3",
        "latex_string": "\\tau_{total} = N \\cdot \\Delta t_{gate} + \\delta_{propagation}",
        "variables_map": {
          "\\tau_{total}": "total_latency",
          "N": "port_count",
          "\\Delta t_{gate}": "gate_reconfiguration_time"
        },
        "bounding_box": {"page": 3, "coords": [120, 500, 400, 550]}
      }
    ],
    "assertions": [
      {
        "assertion_id": "assert_001",
        "subject": "optical:reconfiguration_delay",
        "predicate": "equals",
        "object": "12.5",
        "unit": "ms",
        "context_conditions": {
          "network_load": "80%",
          "topology": "mesh"
        },
        "domain_pack_namespace": "optical",
        "evidence_links": [
          {"source_id": "chunk_p4_c2", "confidence_score": 0.95},
          {"source_id": "form_eq_3", "confidence_score": 0.93}
        ]
```

```
      }
    ],
    "subdomain_assignments": [
      {
        "subdomain_id": "domain.networking.optical.ops",
        "confidence": 0.92,
        "dimension": "switching_mechanism",
        "primary": true,
        "classification_method": "llm"
      },
      {
        "subdomain_id": "domain.networking.optical.satellite",
        "confidence": 0.88,
        "dimension": "application_domain",
        "primary": false,
        "classification_method": "llm"
      }
    ],
    "created_subdomains": ["domain.networking.optical.ops"]
  },
  "id": 2
}
```

2.3 `textDocument/sspAuditConflicts`

- **Direction:** Server $\rightarrow$ Client (Notification)
- **Intent:** Broadcast detected epistemic conflicts when assertions across papers share identical contexts but produce mutually exclusive quantitative/qualitative outputs.
- **Payload:**

```
{
  "jsonrpc": "2.0",
  "method": "textDocument/sspAuditConflicts",
  "params": {
    "conflicts": [
      {
        "conflict_id": "conf_089",
        "metric": "reconfiguration_delay",
        "context_match": {
          "network_load": "80%",
          "topology": "mesh"
        },
        "nodes": [
          {
            "paper_id": "paper_satellite_switching_2026",
```

```
            "assertion_id": "assert_001",
            "value": "12.5 ms",
            "type": "automated_extract"
          },
          {
            "paper_id": "paper_legacy_switch_2023",
            "assertion_id": "assert_104",
            "value": "50.0 ms",
            "type": "automated_extract"
          }
        ],
        "status": "unresolved"
      }
    ]
  }
}
```

2.4 `textDocument/sspResolveConflict`

- **Direction:** Client $\rightarrow$ Server
- **Intent:** Persistent commit log recording a human scientist's manual settlement of a data conflict, declaring a permanent override lock.
- **Request Payload:**

```
{
  "jsonrpc": "2.0",
  "method": "textDocument/sspResolveConflict",
  "params": {
    "conflict_id": "conf_089",
    "resolution": {
      "accepted_assertion_id": "assert_001",
      "overridden_assertion_id": "assert_104",
      "arbitrator_notes": "Paper B used unoptimized FPGA logic; accepted Paper A's ASIC basel
      "lock_status": "permanent_human_lock"
    }
  },
  "id": 3
}
```

2.5 Subdomain Classification System

OntoForge implements **Taxonomy-Based Auto-Classification** to automatically organize papers into research subdomains during ingestion. This addresses the challenge of managing large paper collections

(100+ papers) that span multiple specialized areas within a domain.

2.5.1 Classification Architecture

Multi-Subdomain Assignment: Papers can belong to multiple subdomains simultaneously, each with:

- **Confidence Score** (0.0-1.0): AI-generated certainty of the classification
- **Primary Flag**: Distinguishes main focus from tangential topics
- **Classification Method**: Tracks provenance (`manual` | `llm` | `rule_based` | `embedding`)

Two-Stage Classification Pipeline:

1. **LLM-Based Classification (Primary):** When an LLM client is available, structured prompts analyze paper metadata and text chunks against taxonomy dimensions, returning classifications with reasoning.
2. **Rule-Based Fallback (Secondary):** When no LLM is available, keyword matching against `classification_hints` in the domain pack provides baseline classification (confidence capped at 0.85).

2.5.2 Taxonomy Dimension Structure

Each domain defines orthogonal classification dimensions in the domain-pack config:

```
taxonomy_tree:
  - name: switching_mechanism
    values: [wdm, optical_packet, burst_switching]
    subdomain_map:
      wdm: domain.networking.optical.wdm
      optical_packet: domain.networking.optical.ops
    description: Switching technologies and methods

  - name: topology
    values: [mesh, ring, star, free_space]
    subdomain_map:
      free_space: domain.networking.optical.fso
      mesh: domain.networking.optical.access
    description: Network topology architectures
```

Design Principles:

- Dimensions should be **orthogonal** (independent classification axes)
- Keep **5-10 values per dimension** (not too granular)
- Map only **high-confidence subdomains** (leave ambiguous values unmapped)
- Not all taxonomy values require subdomain mappings

2.5.3 Classification Workflow

During `textDocument/sspIngest`:

1. **Extract Paper Features:** Title, authors, venue, abstract, sample text chunks
2. **Classify Against Dimensions:** For each taxonomy dimension, determine which value(s) apply
3. **Map to Subdomains:** Use `subdomain_map` to convert taxonomy values to subdomain IDs
4. **Auto-Create Subdomains:** If subdomain nodes don't exist in graph, create them automatically
5. **Create BELONGS_TO Edges:** Link paper to subdomains with confidence scores and metadata
6. **Mark Primary Subdomain:** Highest-confidence classification across all dimensions marked as primary

2.5.4 Query Patterns

Get Papers by Subdomain:

```
MATCH (p:PaperNode)-[:BELONGS_TO]->(d:DomainNode {id: "domain.networking.optical.wdm"})
RETURN p.title, p.year
ORDER BY p.year DESC
```

Find Multi-Subdomain Papers:

```
MATCH (p:PaperNode)-[b:BELONGS_TO]->(d:DomainNode)
WHERE d.id STARTS WITH "domain.networking.optical."
WITH p, count(d) as subdomain_count
WHERE subdomain_count > 1
RETURN p.title, subdomain_count
```

Get Primary Subdomain Only:

```
MATCH (p:PaperNode)-[b:BELONGS_TO {primary: true}]->(d:DomainNode)
RETURN p.title, d.name, b.confidence
```

Find Low-Confidence Classifications (HITL Review Queue):

```
MATCH (p:PaperNode)-[b:BELONGS_TO]->(d:DomainNode)
WHERE b.confidence < 0.7 AND b.classification_method = "llm"
RETURN p.title, d.name, b.confidence
ORDER BY b.confidence ASC
```

2.5.5 Manual Reclassification

Users can override AI classifications via `textDocument/sspReclassifyPaper`:

```
{
  "jsonrpc": "2.0",
  "method": "textDocument/sspReclassifyPaper",
  "params": {
    "paper_id": "paper_satellite_switching_2026",
    "new_subdomain_id": "domain.networking.optical.wdm",
    "confidence": 1.0,
    "classification_method": "manual"
  },
  "id": 4
}
```

This creates new BELONGS_TO edges with `classification_method: "manual"` and removes or updates existing auto-generated classifications.

2.5.6 Implementation Status

Completed Components:

- ✅ Enhanced `BelongsToEdge` schema with confidence, primary, classification_method
- ✅ `TaxonomyDimension` schema for structured taxonomy definitions
- ✅ `SubdomainClassifier` service (LLM + rule-based classification)
- ✅ `IngestionService` orchestration with auto-classification
- ✅ Test suite with fixtures and runnable examples
- ✅ Example domain-pack config (`domain-pack-optical-networks.yaml`)
- ✅ Comprehensive documentation (`backend/docs/SUBDOMAIN_CLASSIFICATION.md`)

Integration Points:

- Update `textDocument/sspIngest` handlers to use `IngestionService`
- Add subdomain filter UI components to Matrix View
- Implement `textDocument/sspReclassifyPaper` endpoint
- Create HITL review panel for low-confidence classifications

---

2.6 Intent-Based Interaction Framework

OntoForge treats **all HITL actions as declarations of research intent** rather than isolated corrections. This framework captures, learns from, and proactively applies user preferences across the multi-tier architecture, transforming reactive error handling into predictive assistance.

2.6.1 The Intent Spectrum

Every human intervention falls on a spectrum from tactical fixes to strategic declarations:

```
Tactical Corrections ←————————————→ Strategic Intent
(Fix OCR typo)     (Choose units)     (Prefer ASIC>FPGA)     (Lock baseline)     (Define rese
   Tier 3              Tier 2              Tier 1                Tier 1              Meta-Lev
```

Intent Taxonomy:

Intent Type	Scope	Generalization	Example	AI Learning Potential
Correction	Local	None	"Fix 'latrncy' → 'latency'"	Low (paper-specific)
Preference	Pattern	Medium	"I prefer [ms] over [µs] for this metric"	Medium (context-dependent)
Priority	Pattern	High	"ASIC implementations > FPGA in conflicts"	High (cross-paper rule)
Constraint	Global	Absolute	"This assertion is locked ground truth"	Absolute (never override)
Goal	Global	Strategic	"Build WDM performance comparison matrix"	Shapes entire workflow

2.6.2 Intent Capture Protocol

Enhanced `textDocument/sspResolveConflict` with Intent Metadata:

```
{
  "jsonrpc": "2.0",
  "method": "textDocument/sspResolveConflict",
  "params": {
    "conflict_id": "conf_089",
    "resolution": {
      "accepted_assertion_id": "assert_001",
      "overridden_assertion_id": "assert_104",
      "arbitrator_notes": "Paper B used unoptimized FPGA logic; accepted Paper A's ASIC basel
      "lock_status": "permanent_human_lock",
      "intent_metadata": {
        "intent_type": "priority",
        "intent_scope": "pattern",
        "generalization_hint": "Prefer ASIC over FPGA for reconfiguration_delay baselines",
        "confidence_override": 1.0,
        "tags": ["hardware_preference", "baseline_selection"]
      }
```

```
    }
  },
  "id": 3
}
```

New Method: `textDocument/sspDeclareIntent`

Explicit intent declaration without conflict resolution:

```
{
  "jsonrpc": "2.0",
  "method": "textDocument/sspDeclareIntent",
  "params": {
    "intent_type": "goal",
    "intent_scope": "global",
    "description": "Build comprehensive WDM performance comparison matrix for survey paper",
    "constraints": {
      "focus_subdomains": ["domain.networking.optical.wdm"],
      "time_range": {"min_year": 2020, "max_year": 2026},
      "required_metrics": ["reconfiguration_delay", "cross_talk", "spectral_efficiency"]
    },
    "active_from": "2026-05-24",
    "active_until": "2026-08-15"
  },
  "id": 5
}
```

Response:

```
{
  "jsonrpc": "2.0",
  "result": {
    "intent_id": "intent_wdm_survey_2026",
    "status": "active",
    "affected_systems": [
      "subdomain_filter",
      "conflict_detector",
      "review_queue_prioritization"
    ],
    "recommendations": [
      "21 papers match your criteria (12 need metric extraction)",
      "3 existing conflicts in WDM subdomain await resolution",
      "Suggest prioritizing papers from OFC 2024-2026"
    ]
  },
```

```
  "id": 5
}
```

2.6.3 Intent Learning Engine

Pattern Recognition Queries:

The system continuously analyzes `HumanArbitratorNode` history to extract preferences:

```
// Discover evidence ranking preferences
MATCH (h:HumanArbitratorNode {intent_type: "priority"})-[:LOCKS]->(a:AssertionNode)
WHERE h.timestamp > datetime() - duration('P90D')
WITH h.arbitration_notes as notes,
     h.generalization_hint as hint,
     count(*) as frequency
RETURN notes, hint, frequency
ORDER BY frequency DESC
LIMIT 10

// Output:
// | notes | hint | frequency |
// | "Prefer ASIC over FPGA..." | "hardware_preference" | 23 |
// | "Accept peer-reviewed over arXiv..." | "publication_quality" | 18 |
// | "Trust experimental over simulation..." | "evidence_type" | 15 |
```

Intent Profile Construction:

```python
class UserIntentProfile:
    user_id: str
    learned_priorities: Dict[str, float]  # "asic_over_fpga": 0.87
    preferred_units: Dict[str, str]       # "reconfiguration_delay": "ms"
    evidence_ranking: List[str]           # ["experimental", "simulation", "analytical"]
    active_goals: List[IntentGoal]
    subdomain_focus: List[str]
    temporal_preferences: Dict[str, Any]

    def compute_suggestion_confidence(self, conflict: Conflict) -> float:
        """Calculate confidence for auto-suggesting resolution based on learned intent."""
        pass
```

2.6.4 Proactive Intent Application

AI-Suggested Resolutions:

When conflicts arise, the system proactively suggests resolutions:

```
{
  "jsonrpc": "2.0",
  "method": "textDocument/sspAuditConflicts",
  "params": {
    "conflicts": [
      {
        "conflict_id": "conf_120",
        "metric": "reconfiguration_delay",
        "context_match": {"network_load": "80%", "topology": "mesh"},
        "nodes": [
          {
            "paper_id": "paper_asic_2026",
            "assertion_id": "assert_201",
            "value": "11.2 ms",
            "implementation": "ASIC"
          },
          {
            "paper_id": "paper_fpga_2025",
            "assertion_id": "assert_305",
            "value": "45.0 ms",
            "implementation": "FPGA"
          }
        ],
        "ai_suggestion": {
          "recommended_assertion_id": "assert_201",
          "confidence": 0.87,
          "reasoning": "Based on your 23 previous conflict resolutions (87% preference), you
          "matched_intent_patterns": [
            "hardware_preference_asic_over_fpga",
            "baseline_selection_production_ready"
          ],
          "user_can_override": true
        }
      }
    ]
  }
}
```


One-Click Batch Resolution:


```
{
  "jsonrpc": "2.0",
  "method": "textDocument/sspApplyIntentPattern",
  "params": {
    "pattern_id": "hardware_preference_asic_over_fpga",
    "scope": {
```

```
      "conflict_ids": ["conf_120", "conf_121", "conf_125"],
      "dry_run": false
    }
  },
  "id": 6
}
```

Response:

```
{
  "jsonrpc": "2.0",
  "result": {
    "conflicts_resolved": 3,
    "assertions_locked": 3,
    "pattern_confidence": 0.87,
    "user_review_recommended": false
  },
  "id": 6
}
```

2.6.5 Intent Versioning & Research Context Switching

Intent Branches:

Researchers work on multiple projects with different priorities. Intent branches isolate decision contexts:

```
{
  "jsonrpc": "2.0",
  "method": "textDocument/sspCreateIntentBranch",
  "params": {
    "branch_name": "wdm_survey_paper_2026",
    "description": "For OFC 2026 WDM performance survey",
    "inherit_from": "main",
    "overrides": {
      "focus_subdomains": ["domain.networking.optical.wdm"],
      "temporal_filter": {"min_year": 2020},
      "priority_metrics": ["reconfiguration_delay", "spectral_efficiency"]
    }
  },
  "id": 7
}
```

Context Switching:

```
{
  "jsonrpc": "2.0",
  "method": "textDocument/sspSwitchIntentContext",
  "params": {
    "branch_id": "wdm_survey_paper_2026"
  },
  "id": 8
}
```

All subsequent HITL actions and AI suggestions now operate within this context.

2.6.6 Intent-Aware Query Interface

Natural Language Intent Queries:

```
{
  "jsonrpc": "2.0",
  "method": "textDocument/sspQueryByIntent",
  "params": {
    "query": "Show me papers that contradict my accepted baselines in WDM subdomain",
    "intent_context": "wdm_survey_paper_2026"
  },
  "id": 9
}
```

Response:

```
{
  "jsonrpc": "2.0",
  "result": {
    "papers": [
      {
        "paper_id": "paper_contradicting_2025",
        "title": "Alternative WDM Architecture...",
        "contradiction_type": "value_mismatch",
        "your_baseline": {"assertion_id": "assert_001", "value": "12.5 ms"},
        "their_claim": {"assertion_id": "assert_890", "value": "8.3 ms"},
        "ai_analysis": "Their setup uses custom silicon vs. your COTS baseline",
        "review_priority": "high"
      }
    ],
    "summary": "Found 3 papers with contradictions (2 high priority, 1 low priority)"
  },
```

```
  "id": 9
}
```

Smart Review Queue:

```
{
  "jsonrpc": "2.0",
  "method": "textDocument/sspSuggestReviewQueue",
  "params": {
    "strategy": "intent_aligned",
    "limit": 10
  },
  "id": 10
}
```

Response:

```
{
  "jsonrpc": "2.0",
  "result": {
    "review_items": [
      {
        "type": "low_confidence_classification",
        "paper_id": "paper_ambiguous_2026",
        "reason": "Subdomain classification confidence 0.65 - your past reclassification patt
        "estimated_time": "30 seconds",
        "intent_relevance": 0.92
      },
      {
        "type": "missing_context",
        "assertion_id": "assert_456",
        "reason": "Metric extracted but context incomplete - you've resolved 12 similar cases
        "estimated_time": "1 minute",
        "intent_relevance": 0.88
      }
    ],
    "total_pending": 23,
    "estimated_total_time": "15 minutes"
  },
  "id": 10
}
```

2.6.7 Unified HITL-Intent Mapping

How Existing HITL Boundaries Map to Intent Types:

Tier	HITL Boundary	Intent Type	Intent Scope	Auto-Learning
Tier 4	Source artifact corruption	Correction	Local	None
Tier 3	OCR/LaTeX errors	Correction	Local	Low (spelling patterns)
Tier 3	Visual fallback alerts	Correction	Local	Medium (layout patterns)
Tier 2	Unit selection	Preference	Pattern	High (metric-specific)
Tier 2	Metric name alignment	Preference	Pattern	High (alias mapping)
Tier 2	Missing context completion	Preference	Pattern	Medium (context inference)
Tier 1	Subdomain reclassification	Preference	Pattern	High (taxonomy learning)
Tier 1	Conflict resolution	Priority	Pattern	Very High (evidence ranking)
Tier 1	Assertion locking	Constraint	Global	Absolute (never override)
Meta	Domain pack design	Goal	Global	Strategic (research direction)
Meta	Intent branch creation	Goal	Global	Strategic (project context)

2.6.8 Implementation Recommendations

Phase D Enhancement (Immediate):

1. Add `intent_metadata` fields to `HumanArbitratorNode` (Section 3.1)
2. Implement `textDocument/sspDeclareIntent` endpoint
3. Build intent pattern recognition queries
4. Add `ai_suggestion` field to `sspAuditConflicts` responses
5. Create Intent Profile storage and retrieval

Phase E (Production Hardening):

1. Implement `textDocument/sspCreateIntentBranch` for context isolation
2. Build Intent Learning Engine with pattern matching

3. Add `textDocument/sspApplyIntentPattern` for batch operations
4. Create Smart Review Queue with intent-based prioritization
5. Implement intent versioning and migration

Post-MVP (Advanced Features):

1. Natural language intent queries
2. Multi-user intent reconciliation
3. Intent transfer learning across domains
4. Collaborative intent negotiation
5. Intent explanation and transparency tools

2.6.9 Privacy & Control Guarantees

User Sovereignty Principles:

1. **Transparent Intent Storage:** All learned patterns queryable via `textDocument/sspQueryIntentProfile`
2. **Intent Deletion:** Users can delete specific intent patterns or entire branches
3. **Confidence Thresholds:** Users set minimum confidence for auto-suggestions (default: 0.8)
4. **Opt-Out Granularity:** Disable intent learning per tier, per intent type, or globally
5. **Audit Trail:** Every AI suggestion logged with matched intent patterns for review
6. **Local-First:** Intent profiles stored locally in graph database, never sent to cloud

Example: Query Intent Profile

```
{
  "jsonrpc": "2.0",
  "method": "textDocument/sspQueryIntentProfile",
  "params": {
    "user_id": "scholar_123",
    "detail_level": "full"
  },
  "id": 11
}
```

Response:

```
{
  "jsonrpc": "2.0",
  "result": {
    "user_id": "scholar_123",
    "intent_summary": {
      "total_intents_captured": 127,
      "active_goals": 2,
```

```
      "learned_patterns": 18,
      "confidence_threshold": 0.8
    },
    "top_patterns": [
      {
        "pattern_id": "hardware_preference_asic_over_fpga",
        "confidence": 0.87,
        "applications": 23,
        "last_used": "2026-05-20"
      },
      {
        "pattern_id": "unit_preference_ms_for_delay",
        "confidence": 0.94,
        "applications": 31,
        "last_used": "2026-05-24"
      }
    ],
    "active_branches": [
      {
        "branch_id": "wdm_survey_paper_2026",
        "created": "2026-05-01",
        "intents_in_branch": 8
      }
    ]
  },
  "id": 11
}
```


---

3. Persistent Graph Schema & Properties (Kùzu / GraphDB Definition)

The system treats information as an Evolving Ontology-based Property Graph. The graph database schema must enforce the following definitions natively.

3.1 Node Definitions

`WorkspaceNode`

- **Properties:**
- `id` : `STRING` (Primary Key, e.g., `workspace_5g_backhaul_2026`)
- `name` : `STRING` (Human-readable workspace name)
- `description` : `STRING` (Optional, purpose of this workspace)
- `root_path` : `STRING` (File system path to workspace directory)
- `created_at` : `DATETIME`

- `last_modified_at`: `DATETIME`
- `status`: `STRING` (`"active"` | `"archived"` | `"readonly"`)
- `metadata`: `JSON` (Optional, custom workspace metadata)

Purpose: Top-level container organizing research projects. Each workspace can contain multiple domain packs and maintains isolated research context.

`DomainNode`

- **Properties:**
- `id`: `STRING` (Primary Key, e.g., `domain.networking.optical`)
- `name`: `STRING`
- `namespace`: `STRING` (Workspace-scoped namespace, e.g., `optical`, `wireless`)
- `parent_domain_id`: `STRING` (Optional, null for top-level domains, references parent DomainNode for subdomains)
- `domain_pack_id`: `STRING` (External config identifier)
- `domain_pack_version`: `STRING` (Version from external config)
- `taxonomy_tree`: `JSON` (List of TaxonomyDimension objects with subdomain mappings)
- `core_metrics`: `STRING[]` (Valid metrics with namespace prefix, e.g., `["optical:reconfiguration_delay"]`)
- `classification_hints`: `JSON` (Optional, keywords for rule-based subdomain classification)
- `approval_status`: `STRING` (`"draft"` | `"approved"` | `"deprecated"`) (Human approval of domain taxonomy structure)

`PaperNode`

- **Properties:**
- `id`: `STRING` (Primary Key, derived unique slug or DOI)
- `doi`: `STRING`
- `title`: `STRING`
- `authors`: `STRING[]` (List of author names)
- `year`: `INT`
- `venue`: `STRING` (Publication venue, e.g., conference or journal name)
- `provenance_hash`: `STRING` (SHA-256 fingerprint of the source PDF)
- `import_status`: `STRING` (`"pending"` | `"verified"` | `"rejected"`) (Human verification of source artifact integrity)

`ChunkNode`

- **Properties:**
- `id`: `STRING` (Primary Key, e.g., `paper_slug#chunk_idx`)
- `raw_text`: `STRING`

- `bounding_box` : `JSON` (Map coordinates: `{"page": INT, "box": [x1, y1, x2, y2]}`)
- `verification_status` : `STRING` (`"raw"` | `"under_correction"` | `"pending_verification"` | `"verified"` | `"rejected"`)
- `last_modified_by` : `STRING` (Optional, arbitrator_id who last edited)
- `last_modified_at` : `STRING` (Optional, ISO 8601 timestamp)
- `modification_count` : `INT` (Default: 0, number of edits)

Verification States (HITL Correction Loop):

- `raw` : Machine-extracted, never touched by human
- `under_correction` : Human actively editing (locked by UI)
- `pending_verification` : Edited, awaiting review by different arbitrator
- `verified` : Approved as correct (can be reopened if new errors found)
- `rejected` : Marked as unusable (corrupted OCR, wrong extraction)

`FormulaNode`

- **Properties:**
- `id` : `STRING` (Primary Key)
- `latex_string` : `STRING`
- `variables_map` : `JSON` (Key-value semantic bindings of symbols)
- `constraints` : `JSON` (Boundary conditions under which equations hold true)
- `verification_status` : `STRING` (`"raw"` | `"under_correction"` | `"pending_verification"` | `"verified"` | `"rejected"`)
- `last_modified_by` : `STRING` (Optional, arbitrator_id who last edited)
- `last_modified_at` : `STRING` (Optional, ISO 8601 timestamp)
- `modification_count` : `INT` (Default: 0, number of edits)

Verification States (HITL Correction Loop):

- `raw` : Machine-parsed LaTeX, never touched by human
- `under_correction` : Human actively fixing LaTeX/variable mappings (locked by UI)
- `pending_verification` : Edited, awaiting review
- `verified` : Approved as correct
- `rejected` : Marked as unusable (unparseable, wrong formula)

`AssertionNode`

- **Properties:**
- `id` : `STRING` (Primary Key)
- `subject` : `STRING` (Namespaced metric name, e.g., `optical:reconfiguration_delay`)
- `predicate` : `STRING` (`"equals"` | `"is_bounded_by"` | `"decreases_to"`)
- `object` : `STRING` (Quantitative or qualitative output value)

- `unit`: `STRING`
- `domain_pack_namespace`: `STRING` (Which domain pack this assertion belongs to)
- `context_conditions`: `JSON` (Environmental limits, e.g., load, temperature, topology)
- `status`: `STRING` (`"pending"` | `"resolved"` | `"locked"`)

`HumanArbitratorNode`

- **Properties:**
- `id`: `STRING` (Primary Key, e.g., `arbitrator_user_session_timestamp`)
- `user_id`: `STRING` (Identifier of the human who made the decision)
- `timestamp`: `DATETIME` (When the arbitration was committed)
- `arbitration_notes`: `STRING` (Human rationale for the decision)
- `lock_type`: `STRING` (`"permanent_human_lock"` | `"temporary_override"`)
- `intent_type`: `STRING` (`"correction"` | `"preference"` | `"priority"` | `"constraint"` | `"goal"`) (Classifies the intent level - see Section 2.6.1)
- `intent_scope`: `STRING` (`"local"` | `"pattern"` | `"global"`) (Defines generalization potential)
- `generalization_hint`: `STRING` (Optional, human-readable pattern description: “Prefer ASIC over FPGA for baseline selection”)
- `confidence_override`: `FLOAT` (Optional, user’s confidence in this decision, 0.0-1.0)
- `intent_branch_id`: `STRING` (Optional, references IntentBranchNode for context isolation)
- `tags`: `STRING[]` (Optional, categorization tags: [“hardware_preference”, “baseline_selection”])

`IntentBranchNode`

- **Properties:**
- `id`: `STRING` (Primary Key, e.g., `intent_wdm_survey_2026`)
- `branch_name`: `STRING` (Human-readable name)
- `description`: `STRING` (Purpose of this intent context)
- `user_id`: `STRING` (Owner of this intent branch)
- `parent_branch_id`: `STRING` (Optional, for branching from existing contexts)
- `created_at`: `DATETIME`
- `active_until`: `DATETIME` (Optional, auto-deactivation for time-bound projects)
- `status`: `STRING` (`"active"` | `"archived"` | `"completed"`)
- `overrides`: `JSON` (Context-specific preferences: subdomain filters, temporal ranges, metric priorities)

3.2 Edge Definitions

- `(:WorkspaceNode)-[:CONTAINS_DOMAIN_PACK]->(:DomainNode)` (Links workspace to loaded domain packs)

- `(:WorkspaceNode)-[:CONTAINS_PAPER]->(:PaperNode)` (Papers belong to workspace, can reference multiple domain packs)
- `(:DomainNode)-[:HAS_BRANCH]->(:DomainNode)`
- `(:PaperNode)-[:BELONGS_TO {confidence: FLOAT, primary: BOOL, classification_method: STRING}]->(:DomainNode)` (Edge properties: `confidence` 0.0-1.0 indicates classification certainty, `primary` marks the main subdomain, `classification_method` tracks origin: `"manual"` | `"llm"` | `"rule_based"` | `"embedding"`)
- `(:PaperNode)-[:CITES]->(:PaperNode)`
- `(:PaperNode)-[:CONTAINS]->(:ChunkNode|:FormulaNode)`
- `(:ChunkNode|:FormulaNode)-[:EVIDENCED_BY {confidence_score: FLOAT}]->(:AssertionNode)` (Edge property `confidence_score` represents AI's confidence that this specific evidence supports the assertion, range 0.0-1.0)
- `(:AssertionNode)-[:CONTRADICTS {conflict_id: STRING}]->(:AssertionNode)` (Bi-directional link populated dynamically by the background Audit Engine)
- `(:HumanArbitratorNode)-[:LOCKS]->(:AssertionNode)` (Prevents automated updates to manually resolved assertions)
- `(:HumanArbitratorNode)-[:APPROVES {approval_type: STRING, approved_version: STRING}]->(:DomainNode)` (Tracks human approval of domain structure changes)
- `(:IntentBranchNode)-[:SCOPES]->(:HumanArbitratorNode)` (Links intent decisions to research context branches)
- `(:IntentBranchNode)-[:INHERITS_FROM]->(:IntentBranchNode)` (Intent branch inheritance hierarchy)
- `(:HumanArbitratorNode)-[:MATCHES_PATTERN {pattern_id: STRING, confidence: FLOAT}]->(:HumanArbitratorNode)` (Connects similar intent patterns for learning)

3.3 Design Considerations: Confidence Score Placement

Current Design Decision: Confidence scores are stored as properties on `[:EVIDENCED_BY]` edges rather than on `AssertionNode` to maintain tier separation (Tier 2 semantic claims remain pure; Tier 3 extraction metadata stays at the evidence layer).

Trade-offs & Open Questions:

1. **Aggregation Strategy:**
    - **Question:** When an assertion has multiple evidence links with different confidence scores (e.g., [0.95, 0.60, 0.93]), how should the system compute an overall assertion confidence?
    - **Options:** Minimum (most conservative), average, weighted by evidence type, or no aggregation (show per-evidence only)
    - **MVP Recommendation:** Display per-evidence scores in UI; defer aggregation strategy until user feedback reveals actual needs
2. **Query Complexity:**

- **Impact:** Graph queries must now traverse edge properties to filter or rank by confidence
- **Mitigation:** Index edge properties; cache computed aggregates if needed for performance
- **Benefit:** Enables granular provenance ("this formula supports the claim with 93% confidence, but this text chunk only 60%")

3. **Re-extraction & Versioning:**
   - **Question:** When re-parsing the same PDF with an improved model, should the system:
     - Replace old confidence scores on existing edges?
     - Create parallel edges with version metadata?
     - Store extraction history in a separate audit log?
   - **MVP Recommendation:** Replace scores on re-extraction; add extraction timestamp to edge properties if versioning becomes critical
4. **Low-Confidence Handling:**
   - **Threshold:** Should evidence links below a certain confidence (e.g., < 0.7) be stored but hidden by default?
   - **UX:** How to surface low-confidence extractions for human review without cluttering the Matrix view?
   - **Fallback:** Ties into Section 4.1's Visual Fallback Alert pattern - low confidence triggers HITL intervention
5. **Alternative Considered:**
   - **Option A:** Single `confidence_score` on `AssertionNode` (simpler queries, loses granularity)
   - **Option B:** Separate `ExtractionMetadataNode` (cleaner separation, adds graph complexity)
   - **Current Choice:** Edge properties balance granularity with query simplicity for MVP

Implementation Note: The `sspIngest` response returns `evidence_links` as an array of objects with per-evidence confidence scores. Client implementations must handle this structure when populating the graph.

---

4. Integration, Pipeline Routing & Fault-Tolerant Engine

To handle raw PDF input non-destructively, OntoForge leverages an asynchronous staging ingestion pipeline combined with structural fail-safe fallbacks.

4.1 Tier 3 Ingestion & Visual Fallback Pattern

The background service calls a vision-based parsing model (e.g., **Marker** or **Nougat**) to convert a 2-column PDF into Markdown and standard LaTeX.

- **Fault State:** The layout engine chokes on unreadable scanning noise, corrupted fonts, or non-standard mathematical layouts, yielding malformed formatting or garbled LaTeX.
- **Fault-Tolerant Strategy:** If parsing confidence scores fall below **70%**, the server flags a `Visual Fallback Alert`. It ceases fine-grained symbolic parsing for that region and degrades the node to a `Raw Text Chunk`.
- **UX Recovery:** The frontend displays an **In-line Yellow Squiggle**. When a user hovers over or clicks this block, OntoForge launches the **Digital Microscope view**. Using the bound `bounding_box` pixel metrics, it crops the absolute visual section of the original PDF canvas and displays it side-by-side with the text box. The human updates it in markdown effortlessly, establishing an immediate recovery path.

4.2 Tier 2 Type Validation & Alignment Interceptor

Extracted assertions are serialized into JSON streams using structured schema mapping libraries (e.g., **Instructor** with **Pydantic**).

- **Fault State:** The LLM successfully extracts an asset value (e.g., `12.5`) but drops the unit identifier (`ms`), or leaves a critical environmental variable unassigned due to ambiguous context phrasing.
- **Fault-Tolerant Strategy:** The parser fails Pydantic runtime validation, including domain-pack checks (unknown metric name, invalid unit, missing required context key). The database pipeline intercepts the corrupted node, refusing to insert a structural invalid asset into the graph. It writes it instead into a `Pending_Alignment` buffer table.
- **UX Recovery:** The cell inside the **Global Alignment Matrix** lights up in a muted gray-red hue with an inline alert icon. Hovering over it prompts a localized wizard modal: *"AI extracted value 12.5 but missed the metric unit. Is this: [ms] [us] [ns]?"* Clicking the right unit repairs the structure instantly, flushing the fixed schema straight down into the graph network.

4.3 Tier 1 Epistemic Graph Protection (The Human-Arbitrator Lock)

As backends run incremental updates, newer or larger models might rewrite elements inside the local database, threatening to overwrite manually refined user modifications.

- **Fault State:** A user updates a column value or accepts a specific paper's assertion during a merge audit. The next day, the ingestion engine triggers a re-indexing task on the local directory, threatening to overwrite human edits with older or re-hallucinated AI conclusions.
- **Fault-Tolerant Strategy:** When a user commits a manual change or overrides an automated value, OntoForge creates a `HumanArbitratorNode` and changes the target `AssertionNode.status` to `LOCKED`.
- **Execution Defense:** During any graph-update or merge operation, the graph engine executes a strict priority routing pass. If an incoming node targets a unique identifier carrying a `LOCKED` property state, the update engine **diverts the new data** into a separate `Shadow_Attributes` sub-dictionary. The user's active, verified data layer remains completely uncompromised.

---

5. MVP Implementation Roadmap

To implement this specification with minimum overhead, prioritize the code tasks in this precise sequence. Each phase has clear deliverables, success criteria, and builds on the previous phase's foundation.

5.1 Phase A: Data Contracts + Config Loader

Objective: Establish the type-safe foundation for all data interchange and domain configuration.

Deliverables:

1. **Pydantic Schema Models** (`backend/schemas/`)
   - `nodes.py` : All node types (DomainNode, PaperNode, ChunkNode, FormulaNode, AssertionNode, HumanArbitratorNode, **IntentBranchNode**)
   - `edges.py` : Edge relationship schemas with properties (EVIDENCED_BY with confidence_score, BELONGS_TO with classification metadata, **SCOPES, INHERITS_FROM, MATCHES_PATTERN for intent graph**)
   - `protocol.py` : JSON-RPC request/response models for all SSP methods (**including 7 intent-related methods: sspDeclareIntent, sspCreateIntentBranch, sspSwitchIntentContext, sspQueryByIntent, sspSuggestReviewQueue, sspApplyIntentPattern, sspQueryIntentProfile**)
   - `domain_pack.py` : DomainPackConfig schema with validation rules
   - `intent.py` : **Intent metadata schemas (IntentMetadata, UserIntentProfile, IntentPattern, IntentGoal)**
2. **Domain Pack Loader** (`backend/config/`)
   - `loader.py` : YAML/JSON parser with schema_version compatibility checking
   - `registries.py` : In-memory registries (metric_registry, alias_registry, unit_registry)
   - `validator.py` : Domain-pack validation logic and conflict detection
   - `watcher.py` : File watcher for hot-reload functionality (if watch=true)
3. **Unit Tests** (`tests/schemas/` , `tests/config/`)
   - Schema validation tests (valid/invalid inputs)
   - Config loading tests with sample domain-pack.yaml
   - Registry building and lookup tests
   - Schema version compatibility tests

Technology Stack:

- **Language:** Python 3.11+
- **Schema Validation:** Pydantic v2

- **Config Parsing:** PyYAML or ruamel.yaml
- **File Watching:** watchdog
- **Testing:** pytest

File Structure:

```
backend/
├── schemas/
│   ├── __init__.py
│   ├── nodes.py
│   ├── edges.py
│   ├── protocol.py
│   └── domain_pack.py
├── config/
│   ├── __init__.py
│   ├── loader.py
│   ├── registries.py
│   ├── validator.py
│   └── watcher.py
└── tests/
    ├── schemas/
    └── config/
```

Success Criteria:

- [ ] All node/edge schemas defined with proper types and constraints
- [ ] **IntentBranchNode and intent edge schemas validated**
- [ ] **Intent metadata schemas support intent_type, intent_scope, generalization_hint fields**
- [ ] **Protocol schemas for 7 new SSP intent methods defined**
- [ ] Sample `domain-pack.yaml` loads without errors
- [ ] Registries correctly built from config
- [ ] Invalid configs rejected with clear error messages
- [ ] 100% test coverage on schema validation

Estimated Effort: 2-3 days (+0.5 days for intent schemas)

5.2 Phase B: SSP Server + Neo4j Integration

Objective: Implement a working JSON-RPC server with real graph database storage and subdomain classification.

Status: ✅ **COMPLETED** (94 tests passing, 74% coverage)

Delivered Components:

1. **FastAPI Server** (`backend/server/`)
   - ✅ `main.py` : FastAPI app with JSON-RPC 2.0 support
   - ✅ `jsonrpc_methods.py` : All SSP protocol handlers
   - ✅ Health check and workspace initialization
2. **Neo4j Integration** (`backend/database/`)
   - ✅ `connection.py` : Neo4j driver with testcontainers support
   - ✅ `node_repository.py` : CRUD operations for all node types
   - ✅ `edge_repository.py` : Relationship management with edge properties
   - ✅ JSON serialization for nested properties
3. **Subdomain Classification** (`backend/services/`)
   - ✅ `subdomain_classifier.py` : LLM + rule-based classification engine
   - ✅ `ingestion_service.py` : Orchestrates paper ingestion with auto-classification
   - ✅ Multi-subdomain assignment with confidence scores
   - ✅ Automatic subdomain node creation
4. **Enhanced Schemas** (`backend/schemas/`)
   - ✅ `domain_pack.py` : TaxonomyDimension support with subdomain_map
   - ✅ `edges.py` : BelongsToEdge with confidence, primary, classification_method
   - ✅ All node and edge types with validation
5. **Test Suite** (`backend/tests/`)
   - ✅ 81 unit tests + 13 integration tests (94 total)
   - ✅ Neo4j container-based integration tests
   - ✅ Subdomain classification test suite with examples
   - ✅ 74% code coverage
6. **Configuration** (`backend/configs/`)
   - ✅ `domain-pack-optical-networks.yaml` : Example config with subdomain mappings
   - ✅ Domain pack loader with hot-reload support

Technology Stack:

- **Framework:** FastAPI with JSON-RPC
- **Database:** Neo4j 5 (Docker container)
- **Testing:** pytest + testcontainers-python
- **Classification:** LLM integration ready (OpenAI/Anthropic)

Key Achievements:

- Full subdomain classification system implemented (Section 2.5)

- Papers automatically classified into subdomains during ingestion
- Confidence scores tracked per classification
- Manual reclassification supported via HITL workflows
- Comprehensive documentation in `backend/docs/SUBDOMAIN_CLASSIFICATION.md`

Remaining Integration:

- [ ] Connect real LLM client for production classification (currently uses rule-based fallback)
- [ ] Implement `textDocument/sspReclassifyPaper` endpoint
- [ ] Add subdomain query endpoints for frontend
- [ ] **Implement 7 intent-related SSP method handlers (Section 2.6.2, 2.6.5, 2.6.6, 2.6.9)**
- [ ] **Create `intent_learning_engine.py` service for pattern recognition (Section 2.6.3)**
- [ ] **Add IntentBranchNode CRUD operations to node_repository.py**
- [ ] **Implement UserIntentProfile storage and retrieval**
- [ ] **Build intent pattern matching queries (Cypher queries from Section 2.6.3)**
- [ ] **Add ai_suggestion field support in conflict detection**

Estimated Original Effort: 3-4 days
Actual Status: Complete with extended subdomain classification features
Additional Intent Work: +2-3 days for intent framework backend

---

5.3 Phase C: Matrix Frontend (Electron)

Objective: Build the core UI for visualizing papers vs. metrics in a data grid, with workspace-aware navigation and multi-domain-pack support.

Deliverables:

1. **Electron App Shell** (`frontend/`)
    - `main.js` : Electron main process, window management
    - `preload.js` : Secure IPC bridge
    - `renderer/` : React/Vue frontend code
2. **Workspace Navigation** (`frontend/renderer/components/workspace/`)
    - **`WorkspaceSidebar.tsx` : Tree-based navigation showing workspace → domain packs → papers/subdomains**
    - **`WorkspaceHeader.tsx` : Display active workspace name and metadata**
    - **`DomainPackNode.tsx` : Collapsible tree node for each domain pack**
    - **`DomainPackPapersNode.tsx` : Papers grouped under domain pack**
    - **`DomainPackMetricsNode.tsx` : Metrics list for domain pack**
    - **`AddDomainPackButton.tsx` : Load additional domain packs into workspace**

3. **Matrix View Component** (`frontend/renderer/components/`)
    - `MatrixGrid.tsx` : Main data grid (papers as rows, metrics as columns), **supports domain pack filtering**
    - `MatrixCell.tsx` : Individual cell with value, status indicator, conflict badges, **AI suggestion indicators**
    - **`DomainPackFilter.tsx` : Dropdown to filter matrix by active domain pack(s)**
    - `PaperRow.tsx` : Paper metadata display (authors, year, venue)
    - `MetricColumn.tsx` : Metric header with **namespace prefix** and unit display
    - **`ConflictResolutionPanel.tsx` : Enhanced conflict resolution UI with AI suggestions, reasoning, matched patterns, and intent metadata capture**
4. **Evidence Inspector** (`frontend/renderer/components/`)
    - `DigitalMicroscope.tsx` : Split-pane view showing PDF crop + text
    - `EvidencePanel.tsx` : List of chunks/formulas supporting an assertion
    - `ConfidenceIndicator.tsx` : Visual representation of confidence scores
    - `BoundingBoxOverlay.tsx` : Highlight regions on PDF canvas
5. **Intent Framework UI** (`frontend/renderer/components/intent/`)
    - **`IntentDeclarationModal.tsx` : UI for explicit intent declaration (sspDeclareIntent)**
    - **`IntentBranchSelector.tsx` : Dropdown/switcher for active intent context**
    - **`IntentProfilePanel.tsx` : View learned patterns, confidence thresholds, active branches**
    - **`IntentPatternCard.tsx` : Display individual pattern with delete/edit actions**
    - **`AISuggestionBadge.tsx` : Visual indicator for AI-suggested resolutions with confidence scores**
    - **`IntentMetadataForm.tsx` : Capture intent_type, scope, generalization_hint during HITL actions**
6. **SSP Client** (`frontend/renderer/services/`)
    - `ssp_client.ts` : JSON-RPC client communicating with backend, **with handlers for 7 intent-related methods**
    - `workspace.ts` : Workspace state management, **including active intent branch context**
    - `cache.ts` : Local caching of paper data
    - **`intent_service.ts` : Intent profile management, pattern retrieval, branch switching**

Technology Stack:

- **Desktop Framework:** Electron 27+
- **Frontend:** React 18 + TypeScript
- **UI Library:** Ant Design, Material-UI, or Shadcn/ui
- **State Management:** Zustand or Jotai

- **Data Grid:** TanStack Table (React Table v8) or AG Grid
- **PDF Rendering:** PDF.js or react-pdf
- **Build:** Vite + electron-builder

File Structure:

```
frontend/
├── main.js
├── preload.js
├── renderer/
│   ├── index.html
│   ├── main.tsx
│   ├── components/
│   │   ├── workspace/
│   │   │   ├── WorkspaceSidebar.tsx
│   │   │   ├── WorkspaceHeader.tsx
│   │   │   ├── DomainPackNode.tsx
│   │   │   ├── DomainPackPapersNode.tsx
│   │   │   ├── DomainPackMetricsNode.tsx
│   │   │   └── AddDomainPackButton.tsx
│   │   ├── MatrixGrid.tsx
│   │   ├── MatrixCell.tsx
│   │   ├── DomainPackFilter.tsx
│   │   ├── ConflictResolutionPanel.tsx
│   │   ├── DigitalMicroscope.tsx
│   │   ├── EvidencePanel.tsx
│   │   ├── ConfidenceIndicator.tsx
│   │   └── intent/
│   │       ├── IntentDeclarationModal.tsx
│   │       ├── IntentBranchSelector.tsx
│   │       ├── IntentProfilePanel.tsx
│   │       ├── IntentPatternCard.tsx
│   │       ├── AISuggestionBadge.tsx
│   │       └── IntentMetadataForm.tsx
│   └── services/
│       ├── ssp_client.ts
│       ├── workspace.ts
│       ├── cache.ts
│       └── intent_service.ts
└── package.json
```

Success Criteria:

- [ ] Electron app launches and loads workspace
- [ ] **Left sidebar shows workspace name and tree of domain packs**
- [ ] **Each domain pack node shows papers count, subdomains, and metrics**

- [ ] **Clicking domain pack filters matrix to show only its papers/metrics**
- [ ] **Can expand/collapse domain pack nodes in tree**
- [ ] **Domain pack namespace visible in metric column headers (e.g., "optical:reconfiguration_delay")**
- [ ] Matrix grid displays 3 mock papers with extracted metrics
- [ ] Cells show values with visual status indicators (pending/resolved/locked)
- [ ] **AI suggestions displayed with confidence scores and reasoning**
- [ ] Double-clicking a cell opens Digital Microscope view
- [ ] Digital Microscope shows PDF crop (mock image) side-by-side with text
- [ ] Evidence panel lists supporting chunks/formulas with confidence scores
- [ ] Conflicting assertions highlighted in red/yellow, **with AI suggestion badges**
- [ ] **Intent declaration modal accessible from conflict resolution panel**
- [ ] **Intent branch selector shows active context in header/sidebar**
- [ ] **Intent profile panel displays learned patterns with delete/edit actions**
- [ ] **Confidence threshold slider (0.0-1.0) affects AI suggestion visibility**
- [ ] **HITL actions (conflict resolution, unit selection, subdomain reclassification) capture intent metadata**
- [ ] **Can add new domain pack to workspace via "Add Domain Pack" button**

Estimated Effort: 5-7 days (+2-3 days for intent UI components, +2 days for workspace navigation)

5.4 Phase D: Advanced Queries & Conflict Detection

Objective: Implement conflict detection logic and advanced query capabilities for the Matrix View.

Status: 🔄 **PARTIALLY COMPLETE** (Graph operations complete, conflict detection pending)

Completed:

- ✅ Full graph database integration (Neo4j)
- ✅ Node and edge CRUD operations
- ✅ Subdomain classification and querying
- ✅ Edge properties (confidence scores, classification metadata)
- ✅ Integration tests with real Neo4j container

Deliverables:

1. **Conflict Detection Engine** (`backend/audit/`)
    - `conflict_detector.py` : Find assertions with matching context but different values
    - `context_matcher.py` : Fuzzy matching of context_conditions (80% vs 80.0% vs "80 percent")
    - `notification.py` : Send sspAuditConflicts messages to frontend

2. **Advanced Query Endpoints** (`backend/server/handlers/`)
    - Subdomain distribution queries
    - Multi-subdomain paper listings
    - Low-confidence classification review queues
    - Evidence chain traversal
    - Cross-paper metric comparisons
3. **Real Ingestion Pipeline** (`backend/ingestion/`)
    - Integration with SubdomainClassifier and IngestionService (already built)
    - Conflict detection during ingestion
    - Shadow attributes for locked assertions

Technology Stack:

- **Graph Database:** Neo4j 5 (already integrated)
- **Query Language:** Cypher
- **Classification:** Subdomain system (complete)

File Structure:

```
backend/
├── audit/
│   ├── __init__.py
│   ├── conflict_detector.py
│   ├── context_matcher.py
│   └── notification.py
└── server/handlers/
    ├── ssp_query_subdomains.py
    ├── ssp_get_distribution.py
    └── ssp_reclassify_paper.py
```

Success Criteria:

- [ ] Conflict detector identifies the pre-designed conflicting assertions
- [ ] Frontend receives sspAuditConflicts notification
- [ ] Resolving a conflict creates HumanArbitratorNode and locks assertion
- [ ] Locked assertions cannot be overwritten by subsequent ingestion
- [ ] Subdomain query endpoints return correct filtered results
- [ ] Low-confidence classifications routed to HITL review queue

Estimated Effort: 4-5 days

5.5 Phase E: Config Evolution & Production Hardening

Objective: Add production-ready features for config management, error recovery, and deployment.

Deliverables:

1. **Config Migration System** (`backend/config/`)
   - `migrations/` : Version upgrade scripts (1.0 → 1.1, etc.)
   - `migrator.py` : Detect schema_version and apply migrations
   - `diff_viewer.py` : Show config changes for user review before hot-reload
2. **Lock-Aware Reconciliation** (`backend/audit/`)
   - `reconciler.py` : Compare incoming extraction with locked assertions
   - `shadow_attributes.py` : Store divergent AI outputs in Shadow_Attributes
   - `conflict_resolver.py` : UI for reviewing shadowed vs locked data
3. **Error Recovery** (`backend/fault_tolerance/`)
   - `retry_logic.py` : Exponential backoff for transient failures
   - `fallback_handlers.py` : Graceful degradation when VLM fails
   - `pending_queue.py` : Store failed extractions for manual review
4. **Deployment Packaging**
   - `Dockerfile` : Backend container image
   - `electron-builder.yml` : Desktop app packaging config
   - `deploy/` : Scripts for database backup, restore, migration
   - `docs/` : User manual and troubleshooting guide

Technology Stack:

- **Containerization:** Docker + Docker Compose (optional)
- **Packaging:** electron-builder (Windows/Mac/Linux)
- **Monitoring:** Logging with structlog, sentry for error tracking
- **Distribution:** Auto-update via electron-updater

Success Criteria:

- [ ] Config schema can upgrade from 1.0 → 1.1 without data loss
- [ ] Hot-reload detects config changes and updates registries safely
- [ ] Locked assertions preserved during re-ingestion
- [ ] Shadow attributes accessible via debug panel in UI
- [ ] Electron app packaged as distributable (.dmg, .exe, .AppImage)
- [ ] Backend runs in Docker container with persistent volume

Estimated Effort: 4-5 days

5.6 Implementation Timeline Summary

Phase	Deliverable	Effort	Status	Can Start After
A	Data Contracts + Config Loader	2-3 days	✅ Complete	Immediately
B	SSP Server + Neo4j + Subdomain Classification	3-4 days	✅ Complete	Phase A complete
C	Matrix Frontend	5-7 days	🔄 In Progress	Phase B complete (parallel possible)
D	Advanced Queries & Conflict Detection	4-5 days	🔄 Partial	Phase B complete
E	Config Evolution	4-5 days	⏳ Pending	Phase D complete

Current Status:

- ✅ **Phase A & B:** Fully operational with 94 passing tests, 74% coverage
- ✅ **Subdomain Classification:** Production-ready system with comprehensive documentation
- 🔄 **Phase C:** Frontend structure created, Electron app launch pending
- 🔄 **Phase D:** Graph operations complete, conflict detection engine pending
- ⏳ **Phase E:** Awaiting completion of Phase D

Total MVP Timeline: ~3-4 weeks (original estimate)
Actual Progress: ~60% complete (Phases A & B + subdomain system)

5.7 Technology Decision Matrix

Component	Primary Choice	Alternative	Rationale
Backend Language	Python 3.11+	Rust, Go	Pydantic ecosystem, rapid prototyping
Graph Database	Kùzu	Neo4j, SQLite+graph	Embedded, no server, Cypher support
Frontend Framework	React + TypeScript	Vue 3, Svelte	Ecosystem maturity, type safety

Component	Primary Choice	Alternative	Rationale
Desktop Shell	Electron	Tauri	PDF.js integration, mature tooling
Data Grid	TanStack Table	AG Grid	Free, highly customizable
PDF Rendering	PDF.js	MuPDF	Browser-native, well-documented
State Management	Zustand	Redux, Jotai	Minimal boilerplate, TypeScript-first

---

5.8 Testing Strategy

Unit Tests:

- Pydantic schema validation
- Config loader and registries
- Conflict detection algorithms
- Graph query builders

Integration Tests:

- JSON-RPC request/response flow
- Database CRUD operations
- Full ingestion pipeline (mock PDF → graph)

End-to-End Tests:

- Electron app launches and loads workspace
- User ingests paper, views matrix, resolves conflict
- Config hot-reload updates UI without restart

Test Coverage Target: 80%+ for backend core logic

5.9 Key Decision Points

Before starting implementation, decide:

1. **Graph Database:** Kùzu (embedded) vs Neo4j (client-server)
    - Kùzu: Simpler deployment, single-file database
    - Neo4j: Better query performance, mature ecosystem

2. **Frontend Build Tool:** Vite vs Webpack
    - Vite: Faster dev server, modern defaults
    - Webpack: More plugins, Electron integration examples
3. **Real PDF Parsing:** When to integrate Marker/Nougat?
    - Phase B: Use mock data
    - Phase D: Still mock, focus on graph logic
    - Post-MVP: Integrate real VLM pipeline
4. **Deployment Model:** Desktop-only vs Desktop+Cloud hybrid
    - Desktop-only: Simpler, privacy-first
    - Hybrid: Enable collaboration, remote storage

---

6. Post-MVP Extensions

6.1 Completed Features

✅ Taxonomy-Based Subdomain Classification (Section 2.5)

Status: Production-ready (Implemented May 2026)

Automatic organization of papers into research subdomains using AI-powered taxonomy classification. Addresses the challenge of managing large paper collections (100+ papers) spanning multiple specialized areas.

Key Features:

- Multi-subdomain assignment with confidence scores
- LLM-based classification with rule-based fallback
- Automatic subdomain node creation
- Manual reclassification support
- Comprehensive query patterns

Implementation:

- Enhanced schemas: `BelongsToEdge`, `TaxonomyDimension`
- Services: `SubdomainClassifier`, `IngestionService`
- Test suite with 100% coverage
- Full documentation: `backend/docs/SUBDOMAIN_CLASSIFICATION.md`
- Example config: `domain_pack_optical_networks.yaml`

Usage: Set `auto_classify: true` in `sspIngest` parse_options.

6.2 Future Work

After completing the remaining MVP phases, consider:

- **Real PDF Parsing:** Integrate Marker/Nougat for actual extraction
- **Multi-User Sync:** Operational transforms for collaborative editing
- **Advanced Queries:** Natural language search over graph
- **Export/Import:** BibTeX, RIS, CSV export for interop
- **Plugin System:** Allow domain-specific extractors
- **Version Control:** Git-like branching for research projects